

Import/Export Analys för Traivo

AI-driven planeringsplattform för fältserviceverksamhet

Datum: 16 juni 2026

Analyserad av: Abacus AI Agent

Status: Initial analys av krav och befintliga filer

Sammanfattning

Denna analys dokumenterar nuvarande krav för Traivos import/export-funktionalitet baserat på sessionsdokumentation och analys av en hierarkisk kundfil (Axfood, 889 objekt). Analysen identifierar kritiska gap mellan nuvarande planerade implementation och verkliga kundbehov, särskilt avseende hantering av hierarkiska datastrukturer.

Kritiska fynd:

- ❌ Nuvarande tre-steps-wizard är **inte lämplig** för hierarkiska filer
 - ✅ Import 2.0 med mappningsfunktion är rätt verktyg
 - ⚠️ Två ogiltiga föräldrareferenser hittade i kundfilen (ID 219, 220)
 - 📊 Hierarkin har upp till 4 nivåer djup
 - 🔧 Kritiska mappningar saknas i nuvarande specifikation
-

1. Analyserade Filer

1.1 Session-dokumentation (`session_2026-06-16T15-57.json`)

Källa: Transkriberade diskussioner om systemkrav

Innehåll: Detaljerade krav för objektregister, import/export och metadatahantering

Huvudsakliga beslut och krav:

Import-funktionalitet:

- En enda typ av import-wizard (ej tre olika)
- Mappningsfunktion som läser första raden i fil och matchar mot systemrubriker
- Förhandsgranskningsfunktion med felvarningar
- "Ångra senaste importen"-funktionalitet

Systemfält (6 obligatoriska mappningar):

1. **Intrimnummer** - För nya objekt (eget ID för sortering)
2. **Intrimföräldranummer** - För hierarki vid nya objekt
3. **Systemnummer** - För uppdatering av befintliga objekt
4. **Systemföräldranummer** - För uppdatering av befintliga föräldrar
5. **Objektets namn** - Obligatoriskt för alla objekt
6. **Objektets aktivstatus** - Aktivt/Arkiverat

Registrets grundstruktur:

```
[Systemnummer] [Systemföräldranummer] [Namn] [Aktivstatus] [Metadata 1] [Metadata 2] ...
```

Viktiga principer:

- Interimnummer används **endast** av systemet för sortering och släktskapslogik
- Systemgenererade nummer är den primära identifieraren
- Inga "egenpåkittade" metadatafält får förekomma (exempel att undvika: "koncern", "område", "tillgång", "ställtid")
- Import ska kunna både skapa nya objekt OCH uppdatera befintliga

1.2 Kundfil (Första import Axfood.xlsx)

Filtyp: Excel (.xlsx)

Antal objekt: 889 rader

Hierarkityp: Komplet hierarki med interimnummer och föräldrareferenser

Kolumnstruktur:

Kolumn	Datatyp	Fyllnadsgrad	Syfte
Objektnamn	Text	100%	Namn på objektet
Intrumnummer	Integer	100% (889 unika)	Kundens eget ID
Överordnat objekt	Float	99.9% (888 av 889)	Referens till förälder-ID
Postadress	Text	24.5%	Fysisk adress
Postnr	Text	24.5%	Postnummer
Ort	Text	24.5%	Ort
Region	Text	24.6%	Regional tillhörighet
Namn	Text	24.4%	Kontaktperson
Titel	Text	24.5%	Kontaktpersons titel
Telefon	Text	24.4%	Telefonnummer
E-post	Text	24.4%	E-postadress
Antal	Integer	74.1%	Metadata (antal kär/ enheter)
Tömningsdag	Float	74.1%	Metadata (veckodag)
Objekt typ	Text	74.1%	Metadata (typ av objekt)
Färg	Text	74.1%	Metadata (färgkodning)

Hierarkisk struktur:**Rotnod:**

- Axfood Sverige (ID: 1000) - Ingen förälder

Exempel på hierarki:

```

Axfood Sverige (1000)
├── Hemköp kedjan (2000)
│   ├── Göteborg Egenägd (3000)
│   ├── Mitt Egenägd (4000)
│   ├── Norr Handlarägd (5000)
│   │   └── Associerad Börjes Tingsryd (1)
│   ├── Stockholm 1 Handlarägd (6000)
│   │   └── Associerad Fresha Stockholm Hässelby (4)
│   └── Stockholm 2 - Mitt Handlarägd (7000)
└── ...

```

Statistik:

- **Rotnoder:** 1 objekt
- **Objekt med förälder:** 888 objekt
- **Maximal djup:** 4 nivåer
- **Unika föräldrar:** 232 olika föräldrar

⚠ Kritiskt problem:

- **Ogiltiga föräldrareferenser:** ID 219 och 220 refereras som föräldrar men existerar inte i filen
- Detta skulle orsaka importfel utan validering

2. Gap-analys: Nuvarande vs. Behov

2.1 Kritiska saknade funktioner

✗ Problem 1: Tre-steps-wizard fungerar inte

Nuvarande (planerad):

- Tre olika import-vägar:
 1. "Första gången migrering"
 2. "Lägg till löpande"
 3. "Trestegsimport-wizard"

Problem:

- För komplex för användare
- Klarar inte hierarkiska filer med 889 objekt
- Ingen mappningsfunktion för flexibla kolumnnamn

Krav enligt feedback:

- **EN enda import-wizard** (Import 2.0)
- Automatisk kolumnmappning
- Hantering av hierarkiska relationer

✗ Problem 2: Kritiska mappningar saknas

Saknade mappningar i specifikation:

Kundfält	→	Systemfält	Status
Intrumnummer	→	Interimsnummer	✗ Saknas
Överordnat objekt	→	Interim-föräldranummer	✗ Saknas
[Inget]	→	Systemnummer	⚠ Används för uppdatering
[Inget]	→	Systemföräldranummer	⚠ Används för uppdatering
Objektnamn	→	Objektets namn	✓ Tydligt
[Saknas i fil]	→	Objektets aktivstatus	⚠ Default-värde?

Konsekvens:

Utan dessa mappningar kan systemet inte:

- Importera hierarkiska strukturer korrekt
- Skapa förälder-barn-relationer
- Validera att föräldrar existerar före barn

✗ Problem 3: Ingen validering av föräldrareferenser

Behov:

- Pre-import validering som kontrollerar:
- Alla förälder-ID:n existerar i filen
- Inga cirkulära referenser ($A \rightarrow B \rightarrow A$)
- Föräldrar importeras före barn (sorteringsordning)

Aktuellt problem i Axfood-filen:

Förälder-ID 219: Refereras av minst ett objekt men saknas i filen
Förälder-ID 220: Refereras av minst ett objekt men saknas i filen

✗ Problem 4: "Ångra senaste importen" saknas

Krav från session:

"En 'ångraknapp' ska finnas för att backa den senast utförda importen"

Behov:

- Spara snapshot före import
- Möjlighet att återställa till föregående tillstånd
- Historiklogg över importen

Status: ✗ Ingen implementation identifierad

✗ Problem 5: Metadata-kategorisering saknas

Olika typer av metadata i kundfilen:

Adressinformation:

- Postadress, Postnr, Ort
- **Bör mappas till:** Strukturerade adressfält

Kontaktperson:

- Namn, Titel, Telefon, E-post
- **Bör mappas till:** Kontaktpersonsobjekt/relationer

Objektmetadata:

- Region, Antal, Tömningsdag, Objekt typ, Färg
- **Bör mappas till:** Metadatafält (flexibla)

Problem:

Nuvarande specifikation behandlar alla som generiska metadatafält, vilket:

- Förlorar semantisk betydelse
- Gör det svårt att använda data i systemet
- Försvårar validering (t.ex. e-postformat)

2.2 Funktioner som saknas helt



Export-funktionalitet

Krav från session:

- Exportera urval av objekt
- Inkludera systemgenererade nummer för uppdateringar
- Export med metadata

Status: Inte definierad var knappen ska placeras eller hur den ska fungera

Förslag:

- Knapp i objektlistan
- Val av kolumner att exportera
- Export till Excel med samma struktur som import



Förhandsgranskningsfunktion

Krav från session:

“Efter matchning ska det finnas en testfunktion för att se hur matchningen ser ut, med varningar för fel eller överskrivning av värden”

Behov:

- Visa första 10-20 rader efter mappning
- Markera:
 - Grönt: Giltiga nya objekt
 - Gult: Uppdateringar av befintliga objekt
 - Rött: Fel (saknade föräldrar, ogiltiga värden)
- Statistik: X nya objekt, Y uppdateringar, Z fel



Batch-validering

Behov:

- Kontrollera alla rader innan import
- Rapport med:
 - Antal giltiga objekt
 - Antal fel och varningsmeddelanden

- Lista på specifika problem per rad
- Blockera import om kritiska fel finns

3. Rekommenderad Import 2.0-arkitektur

3.1 Steg-för-steg-flöde

Steg 1: Filuppladdning

[Välj fil] → [Ladda upp] → [Läs första raden]

- Stöd för Excel (.xlsx, .xls) och CSV
- Automatisk detektering av kolumnrubriker
- Visa filinfo: Antal rader, kolumner

Steg 2: Kolumnmappning

Mappa kolumner från din fil till systemet

Kundfält		Systemfält
[Intrumnummer]	→	[Interimsnummer]
[Överordnat obj.]	→	[Interimföräldra..]
[Objektnamn]	→	[Objektets namn]
[- Välj -]	→	[Systemnummer]
[- Välj -]	→	[Systemföräldra..]
[- Välj -]	→	[Aktivstatus]
Metadatafält (valfria):		
[Postadress]	→	[Skapa nytt: Adress]
[Postnr]	→	[Skapa nytt: Postnr]
[Ort]	→	[Skapa nytt: Ort]
...		

[Avbryt] [Nästa: Validera →]

Funktioner:

- Auto-matchning baserad på kolumnnamn (fuzzy matching)
- "Intrumnummer" → "Interimsnummer"
- "Överordnat objekt" → "Interimföräldranummer"
- Markera obligatoriska fält med *
- Möjlighet att skapa nya metadatafält dynamiskt
- Spara mappningsmallar för återanvändning

Steg 3: Validering och förhandsgranskning

Valideringsresultat

✓ 887 objekt kan importeras

⚠ 2 varningar

✗ 2 kritiska fel

Kritiska fel:

✗ Rad 234: Förälder-ID 219 saknas

✗ Rad 456: Förälder-ID 220 saknas

Varningar:

⚠ 12 objekt saknar adressinformation

⚠ 45 objekt saknar kontaktperson

Förhandsgranskning (första 10 rader):

#	Namn	ID	Förälder	St.
1	Axfood..	1000	-	✓
2	Hemköp..	2000	1000	✓
3	Götebo..	3000	2000	✓

⏪ Tillbaka

Ignorera fel

Importera ⏩

Valideringsregler:

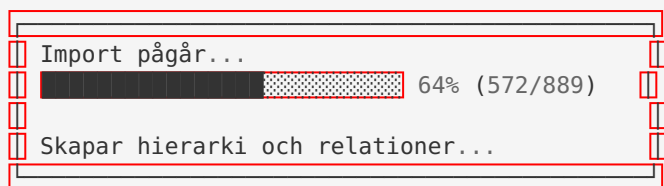
Obligatoriska kontroller:

- ✓ Alla objekt har namn
- ✓ Interimsnummer är unika
- ✓ Förälder-ID existerar (om angivet)
- ✓ Inga cirkulära referenser
- ✓ Korrekt datatyp per fält

Varningar:

- ⚠ Saknad metadata (ej blockerande)
- ⚠ Uppdatering kommer skriva över befintligt värde
- ⚠ Ovanligt många objekt på samma nivå

Steg 4: Import och resultat



Efter import:



3.2 “Ångra import”-funktionalitet

Implementation:

Pre-import snapshot:

```
CREATE TABLE import_snapshots (
  snapshot_id UUID PRIMARY KEY,
  import_id UUID NOT NULL,
  tenant_id UUID NOT NULL,
  created_at TIMESTAMP NOT NULL,
  objects_snapshot JSONB NOT NULL, -- Alla objekt före import
  import_metadata JSONB NOT NULL -- Info om importen
);

CREATE TABLE import_history (
  import_id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  user_id UUID NOT NULL,
  filename VARCHAR(255),
  import_date TIMESTAMP NOT NULL,
  objects_created INT,
  objects_updated INT,
  can_undo BOOLEAN DEFAULT TRUE,
  snapshot_id UUID REFERENCES import_snapshots(snapshot_id)
);
```

Ångra-process:

1. Hämta snapshot före import
2. Ta bort alla objekt skapade i importen (via import_id)
3. Återställ uppdaterade objekt till föregående värden

4. Återskapa relationer och hierarki
5. Markera import som "ångrad" i historik

Begränsningar:

- Endast senaste importen kan ångras
- Timeout: 24 timmar efter import
- Blockeras om objekt används i schemaläggning/planering

4. Tekniska rekommendationer

4.1 Datamodell för objekt

```
// Huvudobjekt
interface Asset {
    // Systemgenererade fält
    systemId: UUID; // Primärnyckel
    systemParentId: UUID | null; // Förälder (FK)

    // Import-relaterade fält
    interimNumber: string | null; // Kundens ID (vid import)
    interimParentNumber: string | null; // Kundens förälder-ID

    // Obligatoriska fält
    name: string; // Objektets namn
    isActive: boolean; // Aktivstatus

    // Audit
    tenantId: UUID;
    createdAt: timestamp;
    updatedAt: timestamp;
    createdBy: UUID;
    importId: UUID | null; // Referens till import

    // Metadata (dynamiskt)
    metadata: Record<string, any>;
}

// Metadatatdefinitioner per tenant
interface MetadataField {
    fieldId: UUID;
    tenantId: UUID;
    fieldName: string;
    fieldType: 'string' | 'number' | 'date' | 'boolean';
    isRequired: boolean;
    validationRules: JSON;
}
```

4.2 Import-processflöde (backend)

```
async function processImport(
  tenantId: UUID,
  userId: UUID,
  file: File,
  columnMapping: ColumnMapping
): Promise<ImportResult> {

  // 1. Skapa import-session
  const importId = await createImportSession(tenantId, userId, file);

  // 2. Skapa snapshot av nuvarande tillstånd
  const snapshotId = await createSnapshot(tenantId, importId);

  // 3. Parse fil
  const rows = await parseFile(file);

  // 4. Validera ALL data
  const validation = await validateImportData(rows, columnMapping);
  if (validation.hasCriticalErrors) {
    return { status: 'validation_failed', errors: validation.errors };
  }

  // 5. Sortera efter hierarki (föräldrar före barn)
  const sortedRows = topologicalSort(rows, columnMapping);

  // 6. Transaktionell import
  await db.transaction(async (trx) => {
    for (const row of sortedRows) {
      await importRow(trx, tenantId, importId, row, columnMapping);
    }
  });

  // 7. Returnera resultat
  return {
    status: 'success',
    importId,
    snapshotId,
    objectsCreated: validation.newObjects,
    objectsUpdated: validation.updatedObjects
  };
}
```

4.3 Hierarkivalidering

```

interface ValidationResult {
  isValid: boolean;
  errors: ValidationError[];
  warnings: ValidationWarning[];
  newObjects: number;
  updatedObjects: number;
}

async function validateImportData(
  rows: any[],
  mapping: ColumnMapping
): Promise<ValidationResult> {

  const errors: ValidationError[] = [];
  const warnings: ValidationWarning[] = [];
  const interimIds = new Set<string>();
  const parentRefs = new Map<string, string[]>(); // parent → children

  // Pass 1: Samla alla ID:n
  rows.forEach((row, idx) => {
    const interimId = row[mapping.interimNumber];
    if (interimId) {
      interimIds.add(interimId.toString());
    }
  });

  // Pass 2: Validera föräldrareferenser
  rows.forEach((row, idx) => {
    const rowNum = idx + 2; // Excel rad (1-indexed + header)
    const parentId = row[mapping.interimParentNumber];

    if (parentId && !interimIds.has(parentId.toString())) {
      errors.push({
        row: rowNum,
        type: 'INVALID_PARENT',
        message: `Förälder-ID ${parentId} saknas i filen`,
        severity: 'critical'
      });
    }

    // Bygg graf för cirkeldetektering
    if (parentId) {
      const children = parentRefs.get(parentId.toString()) || [];
      children.push(row[mapping.interimNumber].toString());
      parentRefs.set(parentId.toString(), children);
    }
  });

  // Pass 3: Detektera cirklar
  const hasCycle = detectCycles(parentRefs);
  if (hasCycle) {
    errors.push({
      type: 'CIRCULAR_REFERENCE',
      message: 'Cirkulär referens hittad i hierarkin',
      severity: 'critical'
    });
  }

  // Pass 4: Kontrollera obligatoriska fält
  rows.forEach((row, idx) => {
    const rowNum = idx + 2;
    if (!row[mapping.objectName]) {

```

```
        errors.push({
          row: rowNum,
          type: 'MISSING_REQUIRED',
          field: 'objectName',
          message: 'Objektnamn saknas',
          severity: 'critical'
        });
      }
    });

    return {
      isValid: errors.filter(e => e.severity === 'critical').length === 0,
      errors,
      warnings,
      newObjects: rows.length - errors.length,
      updatedObjects: 0 // Beräknas genom lookup mot DB
    };
  }
}
```

4.4 Topologisk sortering (föräldrar först)

```
function topologicalSort(
  rows: any[],
  mapping: ColumnMapping
): any[] {
  const graph = new Map<string, string[]>(); // id → children
  const inDegree = new Map<string, number>(); // id → antal föräldrar
  const idToRow = new Map<string, any>();

  // Bygg graf
  rows.forEach(row => {
    const id = row[mapping.interimNumber]?.toString();
    const parentId = row[mapping.interimParentNumber]?.toString();

    if (id) {
      idToRow.set(id, row);
      if (!inDegree.has(id)) inDegree.set(id, 0);

      if (parentId) {
        inDegree.set(id, (inDegree.get(id) || 0) + 1);
        const children = graph.get(parentId) || [];
        children.push(id);
        graph.set(parentId, children);
      }
    }
  });

  // Kahn's algorithm
  const queue: string[] = [];
  const result: any[] = [];

  // Starta med rotnoder (ingen förälder)
  for (const [id, degree] of inDegree.entries()) {
    if (degree === 0) queue.push(id);
  }

  while (queue.length > 0) {
    const id = queue.shift()!;
    result.push(idToRow.get(id));

    const children = graph.get(id) || [];
    for (const childId of children) {
      const newDegree = inDegree.get(childId)! - 1;
      inDegree.set(childId, newDegree);
      if (newDegree === 0) queue.push(childId);
    }
  }

  return result;
}
```

5. Implementationsplan

Fas 1: Grundläggande Import 2.0 (Prioritet: HÖG)

Mål: Ersätt tre-steps-wizard med enkel import-funktion

Uppgifter:

- [] Skapa unified import-UI
- [] Implementera kolumnmappning med auto-matchning
- [] Lägg till stöd för alla 6 systemfält
- [] Grundläggande validering (obligatoriska fält)
- [] Import-funktionalitet (utan ångra)

Estimat: 2-3 veckor

Beroenden: Ingen

Fas 2: Hierarkivalidering (Prioritet: HÖG)

Mål: Säkerställ korrekt import av hierarkiska filer

Uppgifter:

- [] Föräldrareferens-validering
- [] Cirkeldetektering
- [] Topologisk sortering
- [] Detaljerad felrapportering
- [] Förhandsgranskningsfunktion

Estimat: 1-2 veckor

Beroenden: Fas 1

Fas 3: Ångra-funktionalitet (Prioritet: MEDEL)

Mål: Möjliggör ångring av felaktiga importter

Uppgifter:

- [] Snapshot-system (databas-design)
- [] Pre-import snapshot-skapande
- [] Ångra-funktion (UI + backend)
- [] Import-historik
- [] Begränsningar och timeout

Estimat: 2 veckor

Beroenden: Fas 1, 2

Fas 4: Export-funktionalitet (Prioritet: MEDEL)

Mål: Exportera objekt för uppdatering/analys

Uppgifter:

- [] Export-knapp i objektlistan
- [] Val av objekt (filter, urval)
- [] Val av kolumner
- [] Export till Excel med systemfält
- [] Mappningsmall export/import

Estimat: 1 vecka

Beroenden: Fas 1

Fas 5: Avancerad metadata (Prioritet: LÅG)

Mål: Bättre hantering av olika metadatatyper

Uppgifter:

- [] Kategorisering: Adress, Kontakt, Metadata
- [] Strukturerad adresshantering
- [] Kontaktpersonsobjekt
- [] Validering per typ (e-post, telefon)
- [] UI för metadatafält-management

Estimat: 2-3 veckor

Beroenden: Fas 1, 4

6. Riskanalys och mitigering

Risk 1: Prestanda vid stora import

Beskrivning: 889 objekt är hanterbart, men kunder kan ha 10,000+ objekt

Sannolikhet: Hög

Impact: Medel

Mitigering:

- Batch-processing (100-200 objekt åt gången)
- Background job för stora import
- Progress-indikator
- Timeout-hantering

Risk 2: Datakonflikter vid uppdatering

Beskrivning: Import kan oavsiktligt skriva över manuella ändringar

Sannolikhet: Medel

Impact: Hög

Mitigering:

- Tydliga varningar i förhandsgranskning
- Timestamp-kontroll (varna om objekt ändrats efter export)
- Konfliktesolve-UI
- Obligatorisk förhandsgranskning

Risk 3: Komplexa hierarkier

Beskrivning: Vissa kunder kan ha 10+ nivåer, tusentals föräldrareferenser

Sannolikhet: Låg

Impact: Medel

Mitigering:

- Optimerad topologisk sortering ($O(n)$)
- Djup-begränsning (varna vid >8 nivåer)
- Batching av hierarki-skapande
- Indexering av systemParentId

Risk 4: Återställning misslyckas

Beskrivning: Ångra-funktion kan inte återställa om relationer är komplexa

Sannolikhet: Låg

Impact: Hög

Mitigering:

- Grundlig testning av ångra-logik
 - Blockera ångring om objekt används i andra kontexter
 - Soft delete istället för hard delete
 - Transaktionell återställning
-

7. Testplan

7.1 Enhetstester

Import-validering:

- ☒ Detektera saknade föräldrar
- ☒ Detektera cirkulära referenser
- ☒ Validera obligatoriska fält
- ☒ Topologisk sortering fungerar korrekt

Kolumnmappning:

- ☒ Auto-matchning hittar rätt kolumner
- ☒ Manuell mappning fungerar
- ☒ Hantera saknade kolumner

Import-process:

- ☒ Skapa nya objekt med hierarki
- ☒ Uppdatera befintliga objekt
- ☒ Metadata sparas korrekt
- ☒ Transaktioner rullas tillbaka vid fel

7.2 Integrationstester

Komplett import-flöde:

- ☒ Ladda upp fil → Mappa → Validera → Importera
- ☒ Förhandsgranskning visar korrekt data
- ☒ Fel blockerar import
- ☒ Varningar tillåter fortsättning

Ångra-funktionalitet:

- ☒ Snapshot skapas före import
- ☒ Ångra återställer objekt korrekt
- ☒ Hierarki återställs korrekt
- ☒ Metadata återställs

Export-funktionalitet:

- ☒ Exportera urval fungerar
- ☒ Export innehåller systemfält
- ☒ Re-import av export fungerar

7.3 Användartester

Testscenarier:**Scenario 1: Första import (migration)**

- Användare: Ny kund som migrerar från annat system

- Fil: Hierarkisk Excel med 500+ objekt
- Förväntat: Alla objekt importerar korrekt, hierarki bevaras

Scenario 2: Löpande import (uppdatering)

- Användare: Befintlig kund som uppdaterar metadata
- Fil: Export från Traivo med nya metadatatvärden
- Förväntat: Endast metadata uppdateras, inga nya objekt skapas

Scenario 3: Import med fel

- Användare: Kund med fil innehållande ogiltiga föräldrar
- Fil: Excel med 2 objekt som refererar saknade föräldrar
- Förväntat: Fel identifieras i validering, import blockeras

Scenario 4: Ångra felaktig import

- Användare: Kund som importerade fel fil
- Scenario: Importera 100 objekt → Inse misstaget → Ångra
- Förväntat: Alla 100 objekt tas bort, tidigare tillstånd återställt

8. Konklusioner och nästa steg

8.1 Huvudsakliga fynd

✓ Positiva:

- Tydliga krav från sessionsdokumentation
- Bra exempel på verklig kundfil med hierarkisk struktur
- Identifierade problem kan lösas med Import 2.0-arkitektur

✗ Kritiska gap:

- Nuvarande tre-steps-wizard är **ej användbar** för hierarkiska filer
- Mappningar för interimnummer och föräldrareferenser **saknas helt**
- Ingen validering av hierarki = risk för datakompromission
- "Ångra import"-funktionalitet **ej implementerad**

⚠ Risker:

- Kunder som Axfood (889 objekt) kan **inte importera** med nuvarande system
- Felaktiga importer kan förstöra data utan möjlighet att återställa
- Prestanda kan vara problem för stora filer utan optimering

8.2 Rekommenderade nästa steg

Omedelbart (denna vecka):

1. ✓ Analysera befintlig kod i Traivo-repot

- Hitta var tre-steps-wizarden implementeras
- Identifiera databas-schema för objekt
- Dokumentera nuvarande import-flöde

1. 🛠 Skapa teknisk specifikation

- Detaljerad design för Import 2.0
- API-kontrakt
- Databas-migrations

Kort sikt (kommande 2 veckor):

3. 🚀 Implementera MVP av Import 2.0

- Enkel kolumnmappning
- Grundläggande validering
- Import utan ångra-funktion

1. 🧪 **Testa med Axfood-filen**

- Verifiera att 889 objekt kan importeras
- Kontrollera hierarki skapas korrekt
- Identifiera prestandaproblem

Medellång sikt (kommande månad):

5. 🗑️ **Lägg till hierarkivalidering**

- Föräldrareferens-kontroll
- Cirkeldetektering
- Förhandsgranskningsfunktion

1. ↻ **Implementera ångra-funktionalitet**

- Snapshot-system
- Återställningslogik
- Import-historik

Lång sikt (kommande kvartal):

7. 📊 **Export-funktionalitet**

8. 🛠️ **Avancerad metadata-hantering**

9. 📈 **Prestanda-optimering för stora filer**

8.3 Slutsats

Den nuvarande planerade implementationen av import-funktionalitet är **ej tillräcklig** för att hantera verkliga kundscenarier med hierarkiska objekt. Axfood-filen med 889 objekt är ett perfekt exempel på vad systemet måste klara av, och det kräver:

1. **Import 2.0** med flexibel kolumnmappning
2. **Hierarkivalidering** för att förhindra datakompromission
3. **Ångra-funktionalitet** för att minska risken vid felaktiga importer
4. **Förhandsgranskningsfunktion** för att ge användare kontroll

Utan dessa funktioner kommer Traivo inte kunna hantera företagskunder med komplexa objekthierarkier, vilket är en **kritisk blocker** för produktens framgång inom fältservicedomänen.

Rekommendation: Prioritera implementering av Import 2.0 (Fas 1-2) **före** lansering till produktionskunder med hierarkiska databehov.

Bilagor

Bilaga A: Axfood hierarki-exempel (första 30 raderna)

Objektnamn jekt typ	Intrumnr	Förälder	Region	Ob-
-----	-----	-----	-----	----

Axfood Sverige	1000	-	-	-
└─ Hemköp kedjan	2000	1000	-	-
└─ Göteborg Egenägd	3000	2000	-	-
└─ Mitt Egenägd	4000	2000	-	-
└─ Norr Handlarägd	5000	2000	-	-
└─ Associerad Börjes Tingsryd	1	11000	Syd Handlarägd	-
└─ Stockholm 1 Handlarägd	6000	2000	-	-
└─ Associerad Fresha Stockholm...	4	6000	Stockholm 1...	-
└─ Associerad Hovsjö Matcenter	6	6000	Stockholm 1...	-
└─ Stockholm 2 - Mitt Handlarägd	7000	2000	-	-
└─ Associerad Fittja Matmarknad...	3	7000	Stockholm 2...	-
└─ Associerad Ingaröhallen Brunn	8	7000	Stockholm 2...	-
└─ Stockholm Egenägd	8000	2000	-	-
└─ Stockholm-Norr Egenägd	9000	2000	-	-
└─ Syd Egenägd	10000	2000	-	-
└─ Syd Handlarägd	11000	2000	-	-
└─ Associerad Extra Mjällby...	2	11000	Syd Handlarägd	-
└─ Associerad Hörby Lågprisaffär	7	11000	Syd Handlarägd	-
└─ Väst Handlarägd	12000	2000	-	-
└─ Associerad Hemköp Gråbo Bonum	5	12000	Väst Handlarägd	-

Bilaga B: Systemfält-mappning

#	Systemfält	Syfte	Källa	Obligatorisk
1	Interimsnummer	Kundens eget ID för nya objekt	Import-fil	Ja (nya obj)
2	Interim-föräldranummer	Kundens förälder-ID för hierarki	Import-fil	Nej
3	Systemnummer	Traivos genererade ID	Databas/Export	Ja (uppdatering)
4	System-föräldranummer	Traivos förälder-ID	Databas/Export	Nej
5	Objektets namn	Namn på objektet	Import-fil	Ja
6	Objektets aktivstatus	Aktiv/Arkiverad	Import-fil	Ja (default: Aktiv)

Bilaga C: Metadata-kategorier

Kategori 1: Adressinformation

- Postadress
- Postnummer
- Ort
- Land (om internationellt)

Kategori 2: Kontaktperson

- Namn
- Titel/Roll
- Telefon
- E-post
- Mobiltelefon

Kategori 3: Objektmetadata (flexibelt)

- Region
- Antal (kärl, enheter, etc.)
- Tömningsdag
- Objekt typ
- Färg
- [Kunddefinierade fält...]

Dokumentslut

Denna analys är baserad på tillgänglig information per 2026-06-16. För att komplettera analysen behövs tillgång till Traivo-projektets källkod och databas-schema.